C++中函数使用 vector 的完整指南

结合你之前学的 `vector` 基础，我用**最简单、最实用的3种场景**，教你**在函数中使用 vector 数组**的完整操作（包含传参、创建、返回、修改），这是C++开发的标准写法！

核心前提

1. 必须包含头文件：`#include <vector>`
2. 函数中使用 `vector` **无需手动管理内存**，比 `new` 数组安全100倍
3. 传参优先用**引用** `&` （避免拷贝，效率高）

一、3种最常用的函数操作（直接背会）

场景1：函数内部创建vector 并使用

函数里直接定义、赋值、遍历，用完自动释放，零成本。

Code block

```
#include <iostream>
#include <vector>
using namespace std;

// 函数：内部创建vector
void createVector() {
    // 1. 创建空vector
    vector<int> v;
    // 2. 添加元素
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    // 3. 遍历使用
    cout << "函数内vector：";
    for (int num : v) {
        cout << num << " ";
    }
    cout << endl;
} // 函数结束，vector自动释放内存，无需delete

int main() {
```

```
    createVector();
    return 0;
}
```

场景2：函数接收外部vector（重点：传引用）

把 `main` 中的 `vector` 传给函数，**必须用引用 `&`**，否则会拷贝一份新的，修改无效！

① 不修改原vector（只读，用 `const` 引用）

Code block

```
// 只读：const 引用，安全，不修改原数据
void printVector(const vector<int>& v) {
    cout << "遍历vector：";
    for (int i = 0; i < v.size(); i++) {
        cout << v[i] << " ";
    }
    cout << endl;
}
```

② 修改原vector（读写，用普通引用）

Code block

```
// 读写：引用 &，直接修改main中的vector
void modifyVector(vector<int>& v) {
    v.push_back(100);  // 追加元素
    v[0] = 999;        // 修改第一个元素
}
```

场景3：函数返回vector（直接返回，超简单）

`vector` 可以直接作为函数返回值，**不用担心内存问题**（编译器自动优化）。

Code block

```
// 函数：创建并返回vector
vector<int> returnVector() {
    vector<int> v = {10, 20, 30};
    return v;  // 直接返回
```

```
}
```

二、完整可运行代码（整合所有操作）

直接复制运行，覆盖**函数创建、传参、修改、返回**全部用法：

Code block

```
#include <iostream>
#include <vector>
using namespace std;

// 1. 函数内部创建vector
void createVector() {
    vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    cout << "【函数内创建】: ";
    for (int num : v) cout << num << " ";
    cout << endl;
}

// 2. 只读遍历vector（const引用）
void printVector(const vector<int>& v) {
    cout << "【只读遍历】: ";
    for (int num : v) cout << num << " ";
    cout << endl;
}

// 3. 修改外部vector（引用）
void modifyVector(vector<int>& v) {
    v.push_back(100);
    v[0] = 999;
    cout << "【修改完成】" << endl;
}

// 4. 函数返回vector
vector<int> returnVector() {
    vector<int> v = {10, 20, 30};
    return v;
}

int main() {
```

```
    // 调用1：内部创建
    createVector();

    // 调用2：传参、修改
    vector<int> myVec = {10, 20, 30};
    printVector(myVec);
    modifyVector(myVec);
    printVector(myVec);

    // 调用3：接收函数返回的vector
    vector<int> newVec = returnVector();
    cout << "【接收返回值】：";
    for (int num : newVec) cout << num << " ";

    return 0;
}
```

运行结果

Code block

```
【函数内创建】：1 2 3
【只读遍历】：10 20 30
【修改完成】
【只读遍历】：999 20 30 100
【接收返回值】：10 20 30
```

三、函数中使用vector ⚠ 必记注意事项（新手必看）

1. **传参必须用引用 `&`**
   - 错误：`void func(vector<int> v)`（拷贝一份，浪费内存，修改不生效）
   - 正确：`void func(vector<int>& v)`（直接用原vector）
2. **只读操作加 `const`**

`void func(const vector<int>& v)`，防止误修改，更安全。

3. **空vector不能用 `[]` 赋值**

Code block

```
vector<int> v;
v[0] = 10; // ❌ 错误！空数组越界
```

```
v.push_back(10); // ✅ 正确
```

4. **不用手动释放内存**

函数执行完毕，`vector` 自动释放内存，无内存泄漏。

5. **可以存储任意类型**

`vector<string>`、`vector<char>`、`vector<double>` 用法完全一样。

四、极简总结（10秒记住）

1. 函数内用：直接 `vector<int> v` + `push_back`
2. 传参用：`(vector<int>& v)`（修改）/ `(const vector<int>& v)`（只读）
3. 返回用：直接 `return v`
4. 全程不用 `new/delete`，安全高效

（注：文档部分内容可能由 AI 生成）